

گزارش جامع راه اندازی، پایداری و مصورسازی فریم ورک حسگری Micro-ESpectre مبتنی بر ESP32-S3 و پروتکل MQTT

امیرعلی یزدانی

۱۷ شهریور ۱۴۰۵

فهرست مطالب

۱	مقدمه و هدف پروژه	۳
۲	آماده سازی محیط و نصب سرویس MQTT	۴
۱.۲	ساخت فایل پیکربندی سرویس Mosquitto	۴
۲.۲	اجرای کارگزار Mosquitto تحت داکر	۴
۳.۲	تنظیمات فایروال سیستم	۴
۳	فلش و دیپلوی فریم ورک Micro-ESpectre	۴
۱.۳	فلش کردن فریم ورک و آماده سازی گره	۴
۲.۳	تنظیمات فایل پیکربندی	۴
۳.۳	انتقال فایل ها و اجرای پروژه	۴
۴	عیب یابی تخصصی حافظه و پایداری جریان CSI	۴
۱.۴	شرح خطا و بروز مشکل حافظه (ENOMEM)	۴
۲.۴	تحلیل فنی علت بروز خطا	۴
۳.۴	راهکار قطعی: سوئیچ به مد UDP/DNS	۴
۵	مصورسازی داده ها در مانیتور تحت وب	۵
۱.۵	باز کردن داشبورد	۵
۲.۵	تنظیمات اتصال در رابط کاربری	۵
۶	مزیت انتخاب پلتفرم Micro-ESpectre و چشم انداز قابلیت های پیشرفته	۵
۱.۶	شمارش افراد (Human Counting / Sensing)	۶
۲.۶	مرز بندی و مکان یابی فضایی (Zone Detection / Localization)	۶
۳.۶	تشخیص حالت بدنی و ژست (Posture Detection)	۶
۷	تحلیل معماری خط لوله پردازش سیگنال و الگوریتم تشخیص حرکت (MVS Pipeline)	۶
۱.۷	استخراج و پاک سازی ماتریس دامنه (Amplitude Extraction)	۶
۲.۷	فیلترینگ میان گذر و محاسبه واریانس متحرک (Moving Variance)	۷

۷		کالیبراسیون اولیه و تعیین آستانه تطبیقی (Dynamic Thresholding)	۳۰۷
۷		ماشین حالت و صدور خروجی (Decision Engine)	۴۰۷

۱ مقدمه و هدف پروژه

هدف از این پروژه، پیاده‌سازی، پایداری و نمایش زنده داده‌های سیستم حسگری حرکت مبتنی بر اطلاعات وضعیت کانال وای‌فای (WiFi CSI Sensing) با استفاده از بستر نرم‌افزاری **Micro-ESpectre** بر روی میکروکنترلر **ESP32-S3** است. در این سیستم، تغییرات فاز و دامنه امواج بازتابی ناشی از حرکت افراد پردازش شده و نتایج آن جهت پایش وضعیت محیط منتشر می‌گردد.

۲ آماده‌سازی محیط و نصب سرویس MQTT

برای ایجاد بستر تبادل پیام میان برد ESP32-S3 و مانیتورینگ، از سرویس کارگزار Mosquitto استفاده شد.

۱.۲ ساخت فایل پیکربندی سرویس Mosquitto

برای مجاز کردن دسترسی گره‌ها روی پورت استاندارد 1883، فایل کانفیگ اختصاصی ساخته شد:

```
mkdir -p ~/mosquitto_config
cat << 'EOF' > ~/mosquitto_config/mosquitto.conf
listener 1883
allow_anonymous true
EOF
```

۲.۲ اجرای کارگزار Mosquitto تحت داکر

کانتینر کارگزار با دستور زیر اجرا و پورت 1883 نگاشت گردید:

```
sudo docker run -d --name mosquitto --restart unless-stopped -p 1883:1883 -v
~/mosquitto_config/mosquitto.conf:/mosquitto/config/mosquitto.conf
eclipse-mosquitto
```

۳.۲ تنظیمات فایروال سیستم

جهت عبور پکت‌های شبکه از لایه فایروال:

```
sudo ufw allow 1883/tcp
```

۳ فلش و دیپلوی فریم‌ورک Micro-ESpectre

۱.۳ فلش کردن فریم‌ورک و آماده‌سازی گره

جهت استقرار مفسر و فریم‌ورک بهینه‌شده CSI بر روی برد ESP32-S3، حافظه پاک‌سازی و فریم‌ورک اولیه فلش گردید:

```
cd ~/Desktop/Amirali/WifiSensing/espectre/espectre/micro-espectre
source venv/bin/activate
./me flash
```

۲.۳ تنظیمات فایل پیکربندی

مشخصات شبکه و اتصال کارگزار در فایل src/config_local.py تنظیم شد:

```
WIFI_SSID = "Sharif-WiFi"
WIFI_PASSWORD = ""
MQTT_BROKER = "172.27.2.87"
MQTT_PORT = 1883
MQTT_USERNAME = ""
MQTT_PASSWORD = ""
MQTT_TOPIC_PREFIX = "espectre"
```

۳.۳ انتقال فایل ها و اجرای پروژه

با استفاده از ابزار خط فرمان پروژه، سورس کدها به حافظه گره منتقل و برنامه اجرا شد:

```
./me deploy
./me run
```

۴ عیب‌یابی تخصصی حافظه و پایداری جریان CSI

۱.۴ شرح خطا و بروز مشکل حافظه (ENOMEM)

در زمان اجرای اولیه برنامه با ماژول تولید ترافیک پیش فرض (ICMP Ping)، سیستم با خطای زیر متوقف می‌شد:

```
Traffic generator started (ping, 100 pps)
Ping socket error: [Errno 12] ENOMEM
WARNING: Only 0 CSI packets - restarting TG (attempt 2/3)
FATAL: No CSI packets after 3 attempts - cannot operate without traffic
```

۲.۴ تحلیل فنی علت بروز خطا

در پلتفرم MicroPython، فراخوانی سوکت‌های خام (SOCK_RAW) برای ارسال بسته‌های پینگ، به ازای هر بسته به بافرهای پیاپی در حافظه Heap نیاز دارد. همزمانی ورود فریم‌های حجیم داده CSI با این تخصیص‌های مکرر، باعث شکست پیوستگی حافظه شده و سیستم خطای تخصیص حافظه بازمی‌گرداند.

۳.۴ راهکار قطعی: سوئیچ به مد UDP/DNS

جهت حذف کامل سربار سوکت‌های خام، مد تولید ترافیک به بسته‌های استاندارد UDP (DNS Query) تغییر داده شد. در این حالت به علت ساختار سبک سوکت UDP، هیچ‌گونه خطای حافظه‌ای رخ نداد:

```
# src/traffic.py

def __init__(self, mode=MODE_DNS):
```

۵ مصورسازی داده‌ها در مانیتور تحت وب

۱.۵ باز کردن داشبورد

جهت پایش بصری وضعیت، فایل رابط کاربری `espectre-monitor.html` موجود در پوشه پروژه، به صورت مستقیم با دابل کلیک در مرورگر باز شد.

۲.۵ تنظیمات اتصال در رابط کاربری

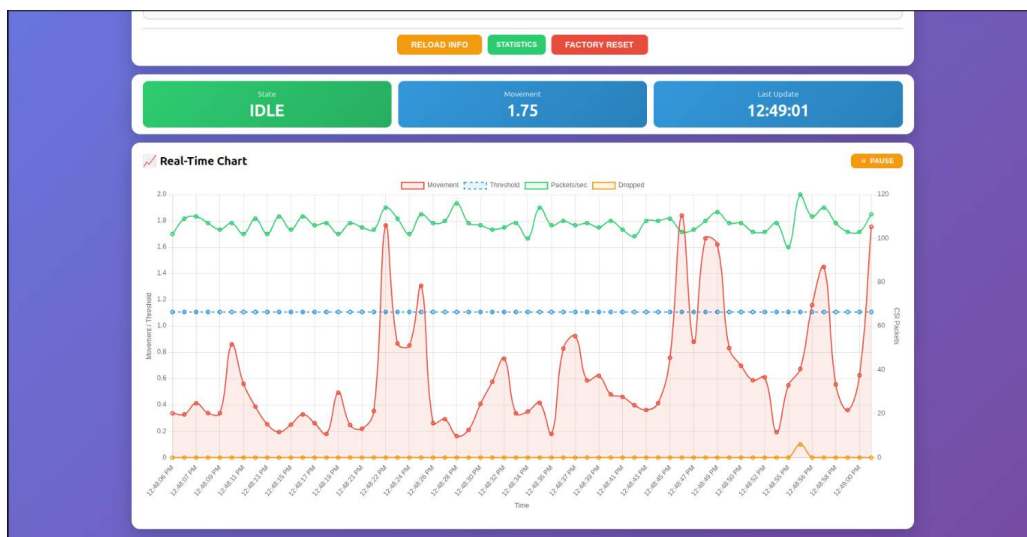
در کادرهای بالای صفحه، مقادیر زیر وارد و بر روی دکمه **CONNECT** کلیک شد:

• **Broker Address:** 172.27.2.87

• **Topic:** espectre/node1

• **Username / Password:** خالی

با برقراری اتصال، نمودار تغییرات دامنه، آستانه حساسیت (Threshold) و تشخیص رخداد حرکت به صورت بلادرنگ نمایش داده شدند.



شکل ۱: نمای داشبورد تحت وب پلتفرم Micro-ESpectre پس از برقراری اتصال و مصورسازی بلادرنگ داده‌ها

۶ مزیت انتخاب پلتفرم Micro-ESpectre و چشم انداز قابلیت‌های پیشرفته

هدف اصلی از راه‌اندازی و تثبیت این بستر، بهره‌گیری از ماهیت مفسری و چابک زبان پایتون در پروژه Micro-ESpectre در مقایسه با فریم‌ورک‌های سخت و کامپایل‌شونده با C/C++ نظیر RuView است. این پلتفرم امکان ایجاد تغییرات سریع در منطق پردازش سیگنال، تست فیلترهای نویز و نمونه‌سازی الگوریتم‌های هوشمند را بدون اتلاف وقت برای کامپایل‌های طولانی میسر می‌سازد. بر پایه این زیرساخت، پیاده‌سازی اهداف زیر در گام‌های بعدی میسر است:

۱.۶ شمارش افراد (Human Counting / Sensing)

- امکان‌پذیری: امکان‌پذیر؛ نیازمند تفکیک تغییرات دامنه امواج ناشی از حضور چندین مانع انسانی مستقل.
- رویکرد پیاده‌سازی: جمع‌آوری دیتاست برچسب‌خورده با حضور تعداد مشخص افراد در محیط، آموزش مدل رگرسیون پایتونی بر روی ماتریس CSI، و فشرده‌سازی مدل نهایی به کمک TensorFlow Lite Micro جهت اجرا روی برد.

۲.۶ مرزبندی و مکان‌یابی فضایی (Zone Detection / Localization)

- امکان‌پذیری: بایک گره منفرد ناممکن است؛ اما با آرایه‌ای از ۲ الی ۳ گره با تکنیک CSI Fingerprinting قابل انجام است.
- رویکرد پیاده‌سازی: ثبت امضای فرکانسی هر بخش از محیط (مانند بخش‌های مختلف اتاق) و دسته‌بندی لحظه‌ای داده‌ها با الگوریتم‌های سبکی نظیر k-NN یا SVM.

۳.۶ تشخیص حالت بدنی و ژست (Posture Detection)

- امکان‌پذیری: یک موضوع تحقیقاتی پیشرفته؛ تفکیک وضعیت‌های نشستن، ایستادن و سقوط نیازمند تحلیل بسیار دقیق فاز زیرحامل‌ها است.
- رویکرد پیاده‌سازی: اعمال شبکه‌های عصبی عمیق (CNN-LSTM) بر روی داده‌های زمانی-مکانی استخراج شده و اجرای بخش استنتاج مدل بر روی سیستم میزبان.

۷ تحلیل معماری خط لوله پردازش سیگنال و الگوریتم تشخیص حرکت (MVS) (Pipeline)

فریم‌ورک Micro-ESpectre از الگوریتم **Moving Variance Segmentation (MVS)** جهت استخراج ویژگی و آشکارسازی حضور و حرکت بهره می‌برد. فرآیند تبدیل فریم‌های خام تا تصمیم‌گیری نهایی در چهار گام اصلی صورت می‌پذیرد:

۱.۷ استخراج و پاک‌سازی ماتریس دامنه (Amplitude Extraction)

بسته‌های دریافتی از درایور وای‌فای حاوی مقادیر مختلط دامنه و فاز (I و Q) برای ۶۴ زیرحامل فرکانسی (Subcarriers) هستند. در نخستین گام، مقدار دامنه هر زیرحامل از رابطه استاندارد استخراج می‌گردد:

$$\text{Amplitude}_k = \sqrt{I_k^2 + Q_k^2} \quad (1)$$

سپس، زیرحامل‌های بدون بار اطلاعاتی (فرکانس مرکزی DC و باندهای محافظ Guard Bands) جهت کاهش حجم محاسبات و حذف نویز از ماتریس حذف می‌شوند.

۲.۰۷ فیلترینگ میان گذر و محاسبه واریانس متحرک (Moving Variance)

به دلیل اینکه حرکات بدن انسان نوساناتی در بازه فرکانسی ۵۰۰ الی ۱۰ هرتز ایجاد می کنند، یک فیلتر دیجیتال میان گذر روی سیگنال اعمال شده تا تغییرات دمایی محیط و نویزهای فرکانس بالای قطعات فیلتر شوند. در ادامه، واریانس لغزان در یک پنجره زمانی متحرک بر روی زیرحامل های منتخب محاسبه می گردد:

$$\text{Var}(t) = \frac{1}{N} \sum_{i=1}^N (A_i - \mu)^2 \quad (2)$$

با تجمع مکانی (Spatial Aggregation) مقادیر واریانس تمام زیرحامل ها، شاخص نهایی حرکت با نام **mvmt** در بازه مقیاس پذیر تولید می گردد. در حالت سکون، مقدار شاخص نزدیک به صفر بوده و با شکست مسیرهای چندگانه امواج (Multipath) ناشی از تحرک افراد، مقدار آن افزایش می یابد.

۳.۰۷ کالیبراسیون اولیه و تعیین آستانه تطبیقی (Dynamic Thresholding)

در ابتدای راه اندازی (فاز دریافت ۱۰۰۰ پکت نخست)، میانگین و انحراف معیار سطح نویز پایه محیط (μ_{noise} و σ_{noise}) ثبت می شود. مقدار آستانه حساسیت (**thr**) بر اساس رابطه زیر کالیبره می گردد:

$$\text{Threshold} = \mu_{\text{noise}} + k \cdot \sigma_{\text{noise}} \quad (3)$$

۴.۰۷ ماشین حالت و صدور خروجی (Decision Engine)

در حلقه اصلی اجرای پایتون، مقدار شاخص حرکت با آستانه کالیبره شده مقایسه شده و وضعیت نهایی به همراه متریک ها در قالب بسته JSON بر روی تاپیک کارگزار MQTT ارسال می گردد:

$$\text{State} = \begin{cases} \text{MOTION}, & \text{fimvmt} \geq \text{thr} \\ \text{IDLE}, & \text{fimvmt} < \text{thr} \end{cases} \quad (4)$$

جهت پایداری خروجی و جلوگیری از نوسانات سریع بین وضعیت ها، یک مکانیزم فیلتر تاخیر زمانی (Hold Time / Debounce) پس از قطع حرکت اعمال شده است.